

Universidad Nacional del Nordeste



Facultad de Ciencias Exactas y Naturales y Agrimensura

Cátedra: Sistemas Operativos

Año:2026

Trabajos Prácticos 2da. Parte

Grupo: 29

Integrantes:

_Navarro Luciano de Jesús 44.683.191

_Romero Enzo Fabian 41948148

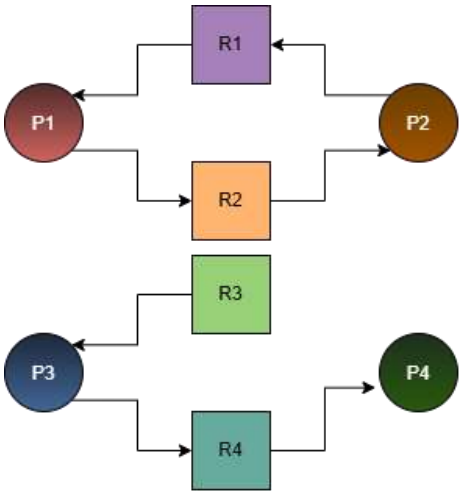
_López Santiago Nicolás 42811857

_Vega Jose Maria 36548306

_Silva Armand Alejandro Omar 36774763

TRABAJO PRÁCTICO N°5: BLOQUEOS

0)



a) ¿Está bloqueado el sistema?

Sí, el sistema está bloqueado porque hay un interbloqueo (*deadlock*) parcial entre los procesos P1 y P2.

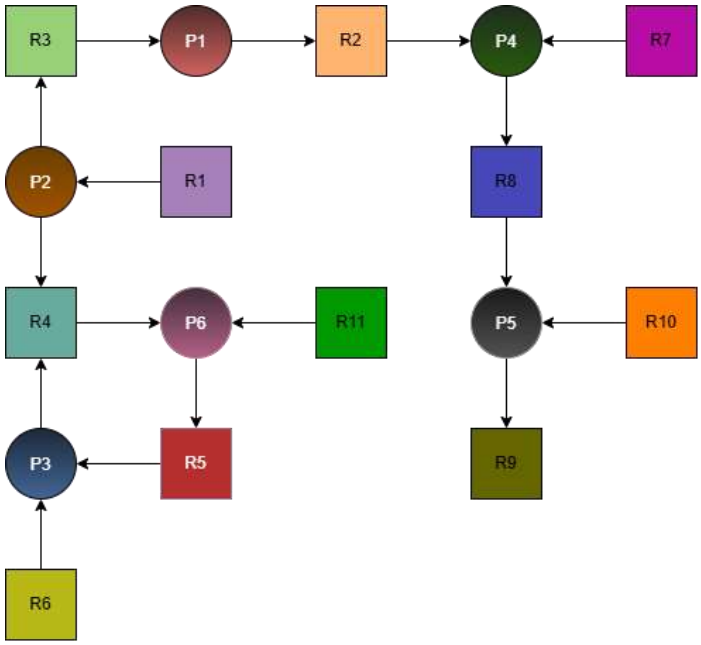
b) ¿Es posible hacer una reducción de la gráfica?

Sí, pero de forma parcial. Se puede iniciar la reducción eliminando primero al proceso P4 (porque no tiene solicitudes pendientes) y luego al proceso P3 (que puede avanzar utilizando el recurso liberado). Sin embargo, la gráfica no se puede reducir por completo porque el algoritmo se detiene al llegar a P1 y P2, ya que al estar atrapados en un ciclo de espera circular ninguno puede terminar ni liberar sus recursos.

c) ¿Cuáles son los procesos bloqueados?

Los procesos que quedan bloqueados de forma permanente son P1 y P2.

1)

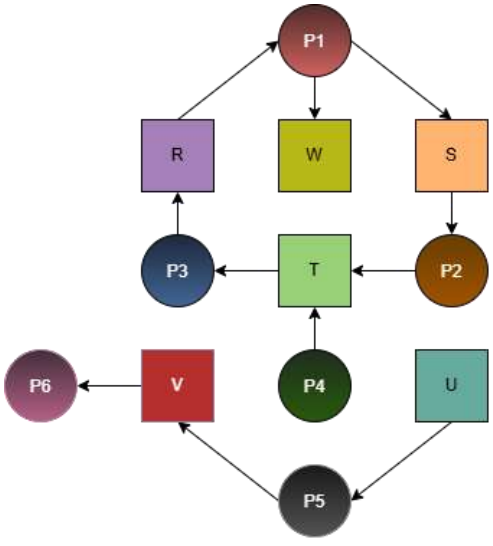


¿Está en interbloqueo este sistema y, en tal caso, cuáles son los procesos involucrados? Mediante la apropiación de que único recurso podría solucionarse el interbloqueo. Describa la secuencia de ejecución de procesos una vez realizada la apropiación.

Sí, el sistema se encuentra en interbloqueo, y los procesos involucrados en el ciclo de espera circular que provocan esta situación son P3 y P6.

El interbloqueo se puede solucionar mediante la apropiación de un único recurso, que es el R4, quitándoselo temporalmente al proceso P6 para asignárselo a P3. Una vez realizada la apropiación y entregado R4 a P3, este completa sus recursos, finaliza su ejecución y libera R5 y R6. A partir de ahí, P6 puede adquirir R5, finalizar y liberar R4 y R11, lo que destraba de manera secuencial al resto de los procesos para que puedan concluir su trabajo.

2)

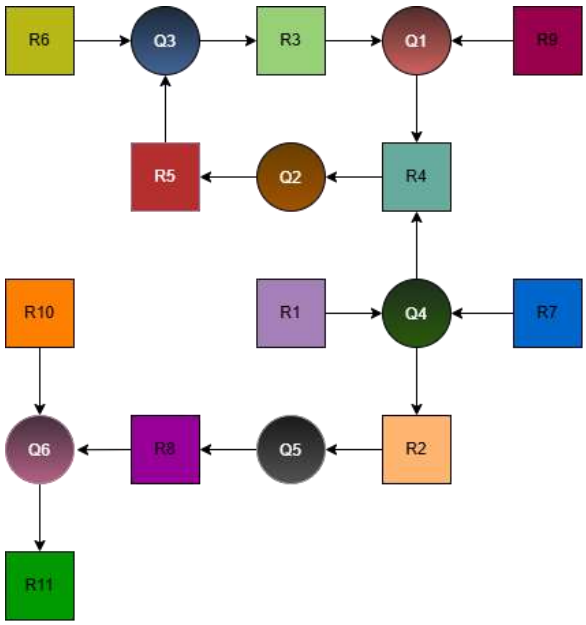


¿Está el sistema en interbloqueo? ¿Hay algún proceso que, sin estar en el ciclo, quede bloqueado igual? Indique cómo recuperar el sistema apropiándose de un único recurso, y la secuencia en la que terminarían los procesos.

Sí, el sistema se encuentra en interbloqueo debido a la presencia de un ciclo de espera circular formado entre los procesos P1, P2 y P3.

Sí, el proceso P4 queda bloqueado a pesar de no estar en el ciclo principal, ya que solicita el recurso T, el cual está retenido por P3 y nunca será liberado. El sistema se puede recuperar mediante la apropiación del recurso T (quitándoselo temporalmente a P3 y dándoselo a P2). La secuencia de finalización es: P2 completa sus tareas y libera S, luego P1 adquiere S y libera R, después P3 obtiene R y libera T, y finalmente P4, P5 y P6 concluyen sus ejecuciones de forma ordenada.

3)

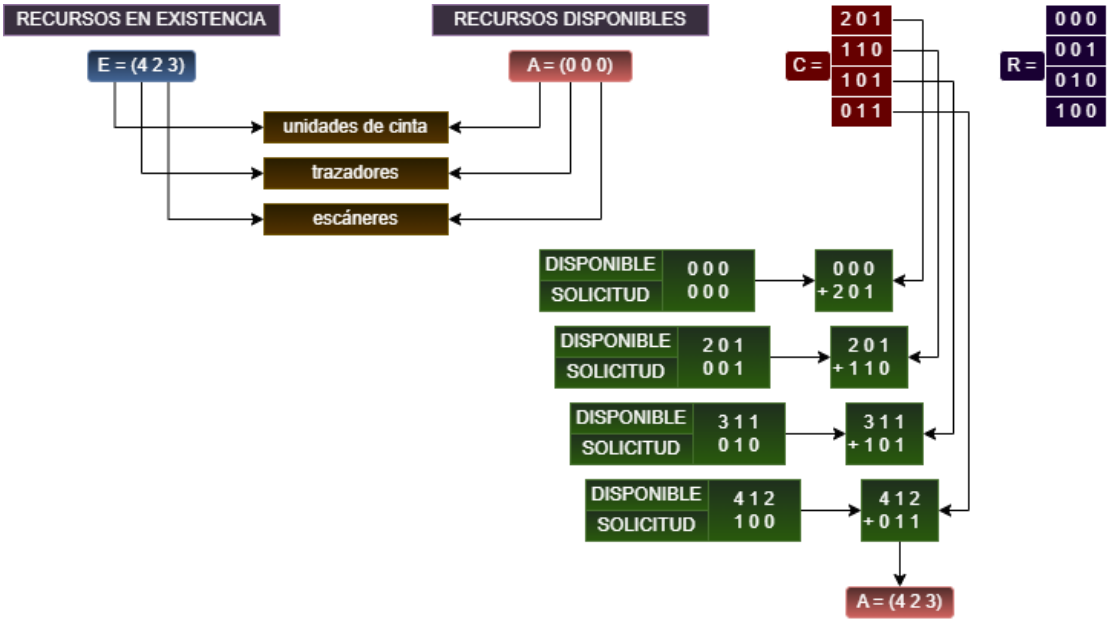


¿Está el sistema en interbloqueo? ¿Qué procesos están realmente en el ciclo y cuálesquedan bloqueados sin estarlo? Indique cómo recuperar el sistema apropiándose de un únicorecurso, y la secuencia en la que terminarían los procesos.

Sí, el sistema está en interbloqueo por un ciclo entre Q1, Q2 y Q3. Los procesos Q4 y Q5 quedan bloqueados sin estar en el ciclo. Se recupera apropiándose del recurso R4

(sacándoselo a Q2 y dándoselo a Q1), lo que permite que terminen Q1, Q3, Q2, Q4, Q5 y Q6 en ese orden.

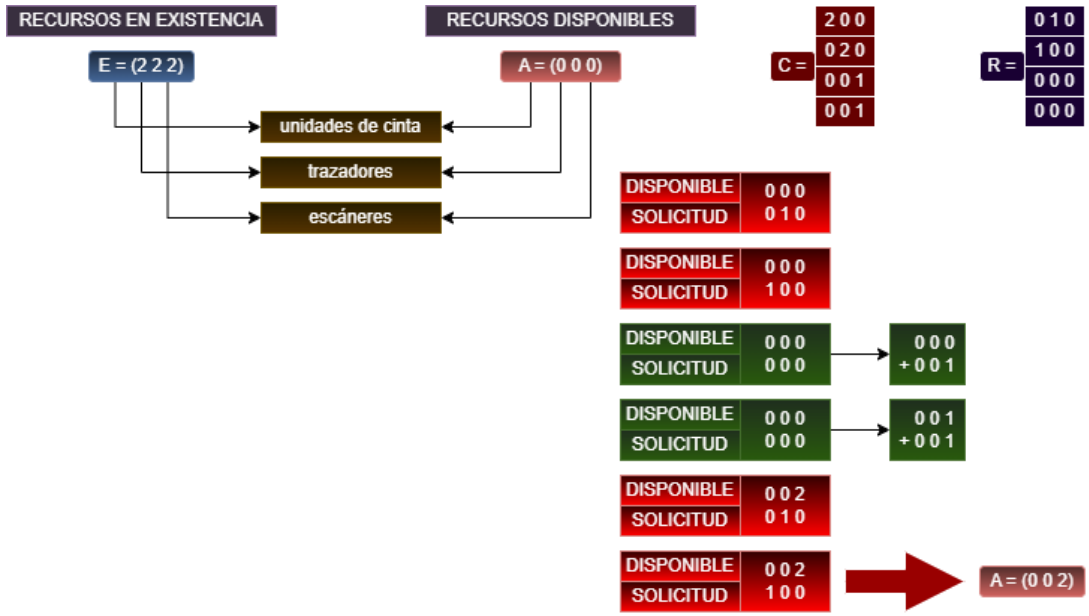
4) No, el sistema no se encuentra en interbloqueo. Aplicando el algoritmo paso a paso, se verifica que cada proceso puede satisfacer su solicitud con los recursos disponibles (o los que van liberando los procesos al terminar), lo que permite que todos concluyan su ejecución de forma secuencial sin que ninguno quede retenido.



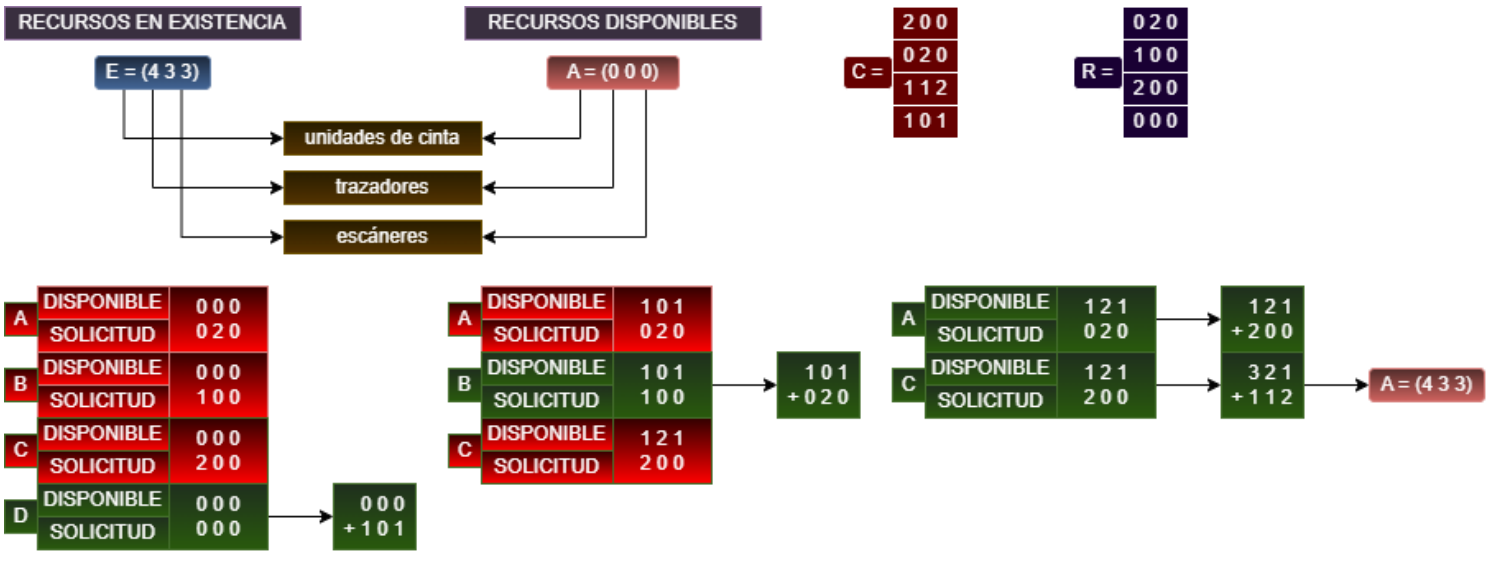
5) Sí, el sistema se encuentra en interbloqueo, ya que ningún proceso puede satisfacer su solicitud con los recursos disponibles actuales y no es posible liberar recursos para que avancen.



6) sí hay interbloqueo parcial, donde C y D pueden terminar, mientras que A y B quedan bloqueados sin poder avanzar.

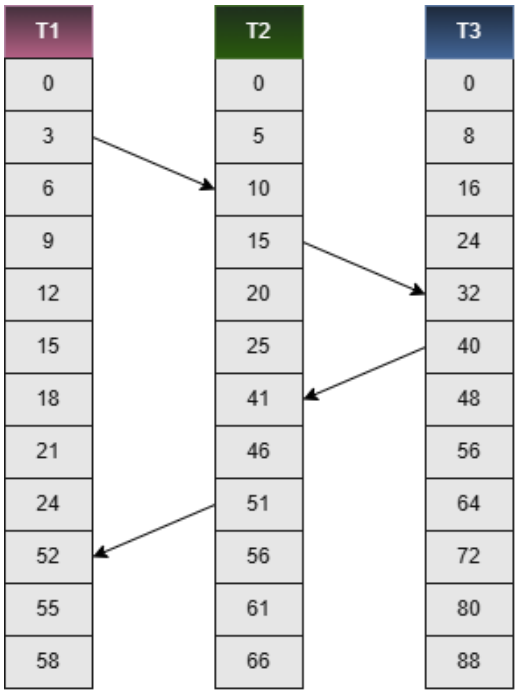


7) El sistema no se encuentra en interbloqueo. El orden en que se van marcando y completando los procesos al aplicar el algoritmo paso a paso es D, B, A y C. El proceso D arranca con su solicitud en cero, termina y libera sus recursos, lo que permite que luego puedan satisfacer sus solicitudes y concluir de manera secuencial los procesos B, A y C sin que ninguno quede retenido.

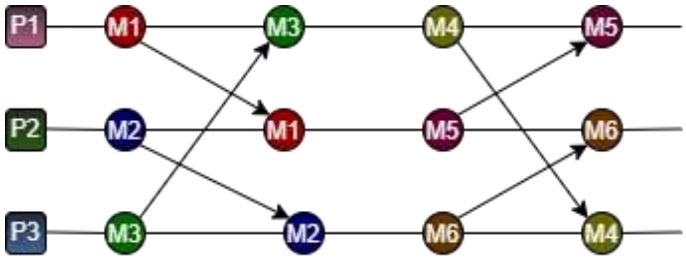


TRABAJO PRÁCTICO N°6: SINCRONIZACIÓN EN SISTEMAS DISTRIBUIDOS

1)

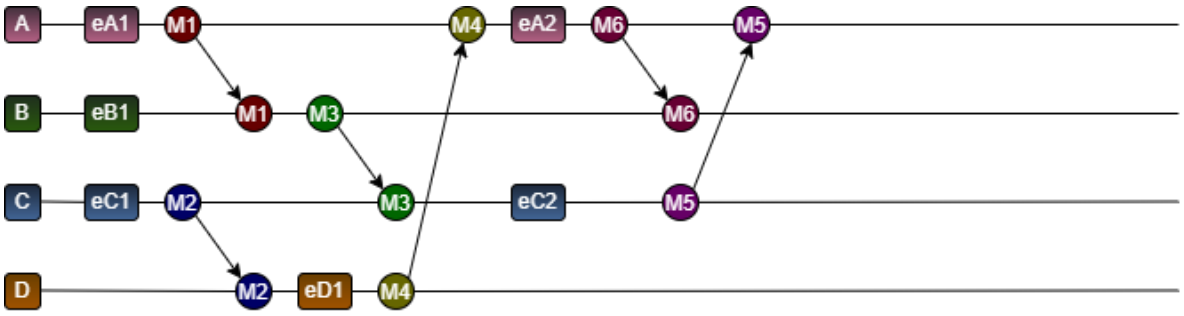


2)



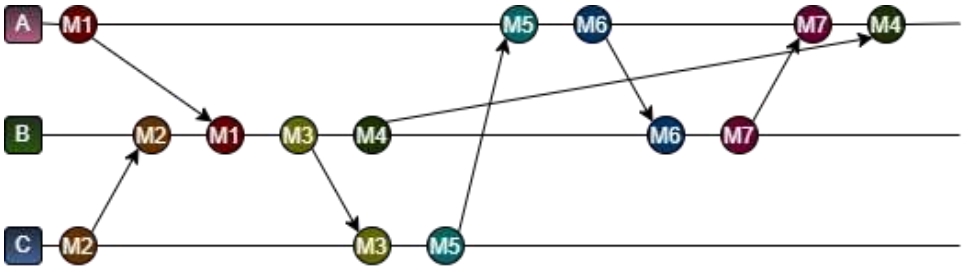
PROCESO	EVENTO	RELOJ
P1	envía M1	1
P1	recibe M3	2
P1	envía M4	3
P1	recibe M5	4
P2	envía M2	1
P2	recibe M1	2
P2	envía M5	3
P2	recibe M6	4
P3	envía M3	1
P3	recibe M2	2
P3	envía M6	3
P3	recibe M4	4

3)



PROCESO	EVENTO	RELOJ
A	evento interno eA1	1
A	envía M1	2
A	recibe M4	6
A	evento interno eA2	7
A	envía M6	8
A	recibe M5	9
B	evento interno eB1	1
B	recibe M1	2
B	envía M3	3
B	recibe M6	9
C	evento interno eC1	1
C	envía M2	2
C	recibe M3	4
C	evento interno eC2	5
C	envía M5	6
D	recibe M2	3
D	evento interno eD1	4
D	envía M4	5

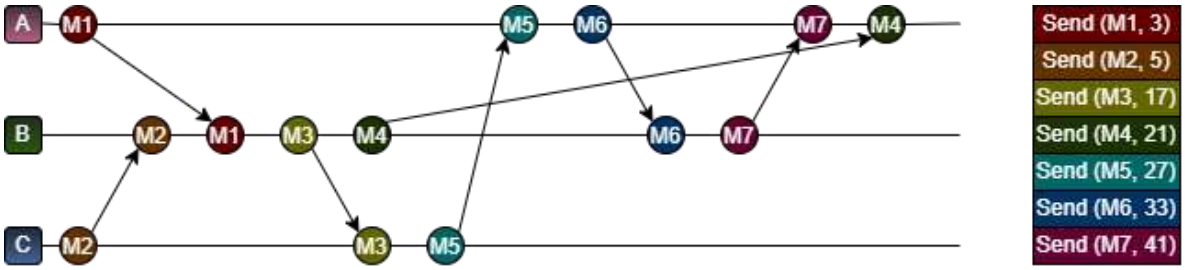
4) a) b)



Send (M1, 3)
Send (M2, 5)
Send (M3, 12)
Send (M4, 16)
Send (M5, 15)
Send (M6, 9)
Send (M7, 24)

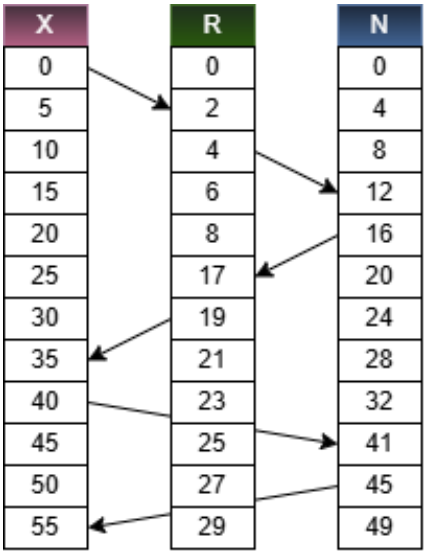
c) El reloj lógico de A marca 15, el de B 24 y el de C 15.

d) e)

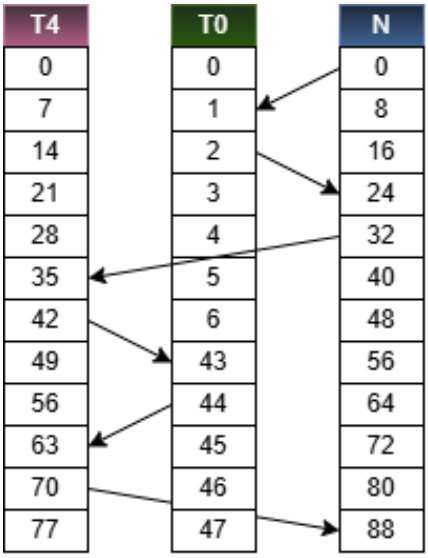


f) El reloj lógico de A marca 47, el de B 41 y el de C 27.

5)



6)



7) Ventajas y desventajas de los sistemas distribuidos (vs. centralizados)

Ventajas

- **Escalabilidad:** Permite aumentar la capacidad del sistema de forma gradual añadiendo nuevos ordenadores (escala horizontal), en lugar de tener que cambiar a un servidor central más caro.
- **Tolerancia a fallos:** Si un nodo o servidor falla, el resto del sistema puede seguir funcionando y prestando servicio sin caerse por completo.
- **Compartición de recursos:** Facilita que múltiples usuarios geográficamente distantes accedan de forma concurrente a hardware, software o datos comunes.

Desventajas

- **Complejidad del software:** Desarrollar, depurar y mantener sistemas distribuidos es mucho más difícil debido a la concurrencia, la sincronización y la gestión de fallos de red.
- **Problemas de red:** El rendimiento depende directamente del ancho de banda y la latencia de la red, sumado al riesgo de que ocurran pérdidas de mensajes o particiones de red.
- **Seguridad:** Al haber múltiples nodos y canales de comunicación expuestos, la superficie de ataque aumenta drásticamente, haciendo más difícil proteger los datos.

8) Concepto y tipos de "transparencia"

La **transparencia** es la capacidad de un sistema distribuido para ocultar al usuario y al programador la separación física de sus componentes, haciendo que todo el entorno se perciba y opere como si fuera **un único sistema centralizado**.

Tres tipos de transparencia

- **Transparencia de localización:** El usuario no sabe en qué servidor o máquina física real están almacenados los recursos (archivos, bases de datos) que está utilizando.
- **Transparencia de replicación:** Oculta el hecho de que existen múltiples copias de un mismo recurso distribuidas en la red para mejorar el rendimiento o la fiabilidad.
- **Transparencia de concurrencia:** Permite que varios usuarios compartan y accedan a los mismos recursos de forma simultánea sin notar interferencias ni dejar el sistema en un estado inconsistente.

Importancia

Es fundamental porque **reduce la carga cognitiva** del usuario y simplifica drásticamente el desarrollo de aplicaciones. Permite que el sistema evolucione, se repare o se amplíe internamente sin que el software final o el usuario tengan que modificar su forma de trabajar.

9) Escalabilidad y cuellos de botella

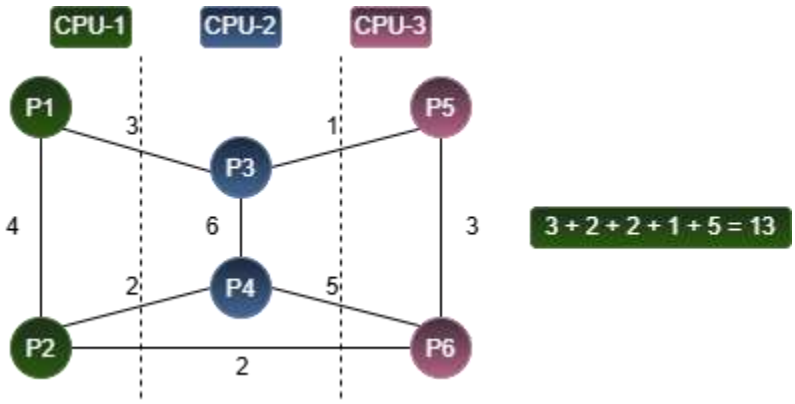
La **escalabilidad** es la propiedad de un sistema distribuido que determina su capacidad para manejar un volumen de trabajo creciente (más usuarios, más datos, más peticiones) añadiendo recursos de hardware de manera proporcional y eficiente.

Tres cuellos de botella potenciales a evitar

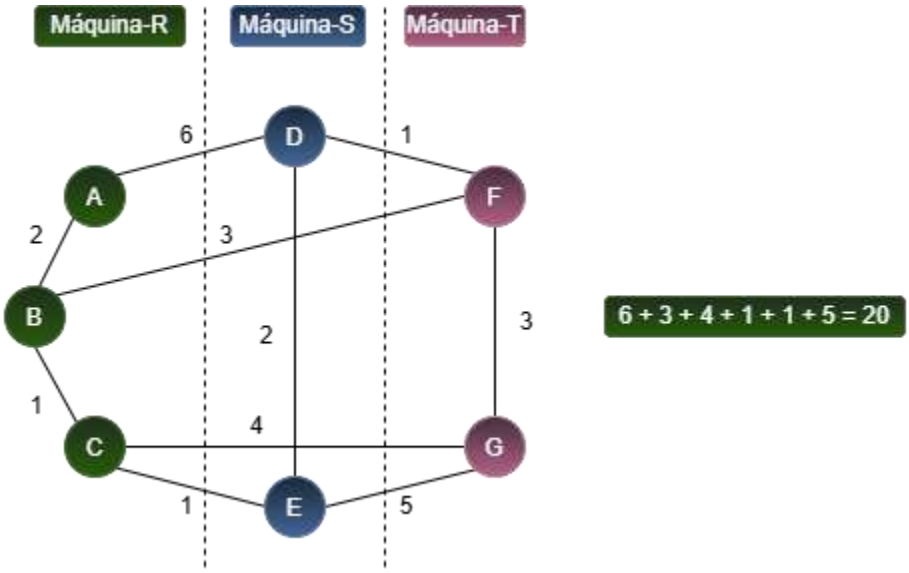
- **Componentes centralizados:** Utilizar un único servidor central para autenticar usuarios o procesar todas las solicitudes satura el hardware de esa máquina a medida que crece el sistema.
- **Datos centralizados:** Mantener una única base de datos central o un solo archivo de registro obliga a todos los nodos a esperar por turnos para leer o escribir, frenando el rendimiento global.
- **Algoritmos centralizados:** Tomar decisiones de enrutamiento o sincronización basadas en un único nodo que recolecta toda la información genera retrasos masivos y latencia por el tráfico acumulado en ese punto.

TRABAJO PRÁCTICO N°7: PROCESOS Y PROCESADORES EN SISTEMAS DISTRIBUIDOS

I) 1) a) b)



2) a) b)



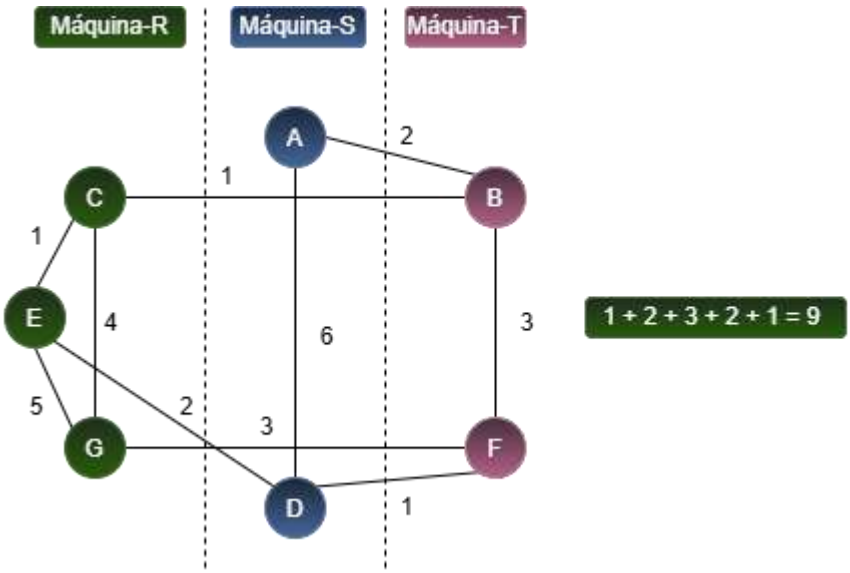
c) Nuevas asignaciones:

_Máquina-R: C, E, G

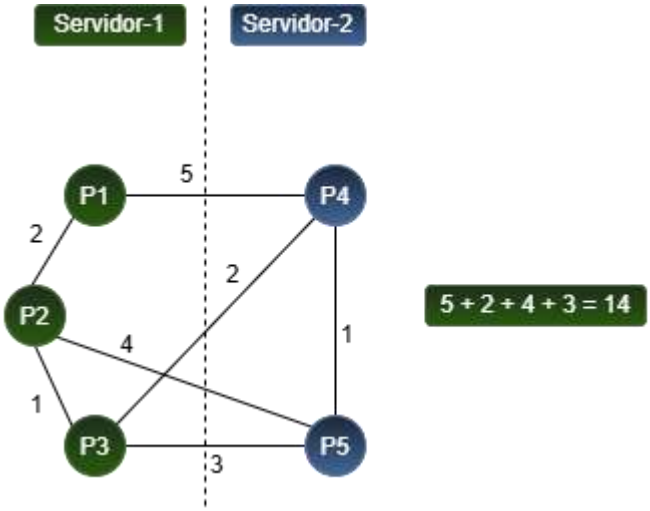
_Máquina-S: A, D

_Máquina-T: B, F

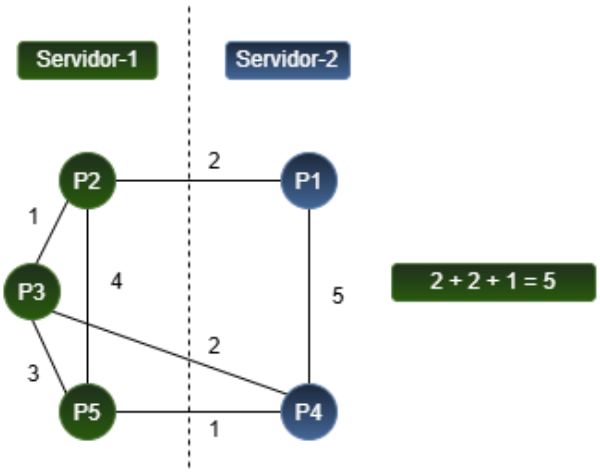
Se agruparon los procesos con mayor tráfico mutuo en las mismas máquinas (como A-D en la Máquina-S, y el grupo C-E-G en la Máquina-R) para maximizar el tráfico local y reducir el uso de la red.



3) a) b)

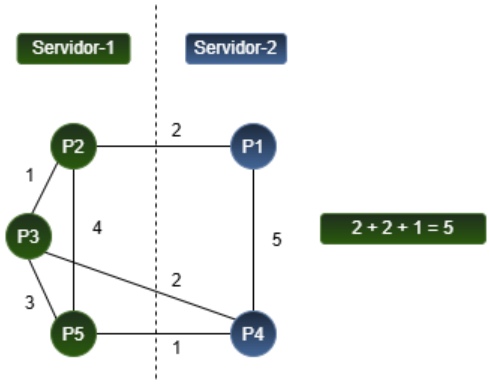


c)



opción	servidor-1	servidor-2	enlaces	tráfico total
1	P3-P4-P5	P1-P2	5+1+4	10
2	P2-P4-P5	P1-P3	2+5+1+2+3	13
3	P2-P3-P5	P1-P4	2+2+1	5
4	P2-P3-P4	P1-P5	2+5+4+3+1	15
5	P1-P4-P5	P2-P3	2+4+2+3	11
6	P1-P3-P5	P2-P4	2+5+1+4+2+1	15
7	P1-P3-P4	P2-P5	2+1+5+2+1	11
8	P1-P2-P5	P3-P4	5+1+3+1	10
9	P1-P2-P4	P3-P5	1+2+4+1	8
10	P1-P2-P3	P4-P5	5+4+2+3	14

II) 4) a)



opción	CPU-1	CPU-2	CPU-3	enlaces	tráfico total
1	P1-P2	P3-P4	P5	6+4=10	23 - 10 = 13
2	P3-P4	P1-P2	P5	4+6=10	23 - 10 = 13
3	P1-P2	P3-P5	P4	6+2=8	23 - 8 = 15
4	P3-P5	P1-P2	P4	2+6=8	23 - 8 = 15
5	P1-P2	P4-P5	P3	6+5=11	23 - 11 = 12
6	P4-P5	P1-P2	P3	5+6=11	23 - 11 = 12
7	P1-P3	P2-P4	P5	2+1=3	23 - 3 = 20
8	P2-P4	P1-P3	P5	1+2=3	23 - 3 = 20
9	P1-P3	P4-P5	P2	2+5=7	23 - 7 = 16
10	P4-P5	P1-P3	P2	5+2=7	23 - 7 = 16
11	P1-P4	P2-P3	P5	3	23 - 3 = 20
12	P2-P3	P1-P4	P5	3	23 - 3 = 20
13	P1-P4	P2-P5	P3	0	23
14	P2-P5	P1-P4	P3	0	23
15	P1-P5	P2-P4	P3	1	23 - 1 = 22

b) **b. Nueva asignación óptima (sin CPU-3):**
Al salir de servicio la CPU-3, la distribución que minimiza el tráfico de red es:

- **CPU-1 (admite 3):** P1 - P2 - P3
- **CPU-2 (admite 2):** P4 - P5

Cálculo del Tráfico de Red:

- Enlaces internos (locales): P1-P2 (6) + P1-P3 (2) + P2-P3 (3) + P4-P5 (5) = 16
- Tráfico de red restante: 23 - 16 = **7**

c) El tráfico en la red disminuye (pasa de 12 a 7) porque al aumentar la capacidad de la CPU-1 a 3 procesos, se permite la cohabitación de un grupo de procesos con un intercambio de datos muy alto (P1, P2 y P3). Al estar juntos en el mismo procesador, su comunicación pasa a ser local (por memoria interna) y deja de viajar por el cable físico de la red, reduciendo el tráfico global del sistema.

5) a) No, ya no es válida. La asignación óptima del ejercicio anterior agrupaba a los procesos {P1, P2} en una misma CPU. Si sumamos sus requerimientos de hardware:

_CPU requerida: 50%(P1) + 60%(P2) = 110%

Como cada CPU admite un máximo de 100% de CPU, el grupo {P1, P2} produce una sobrecarga del procesador, haciendo que esa distribución sea físicamente imposible.

b) **Pares de procesos que no pueden compartir una CPU:**

Para que dos procesos puedan compartir una CPU, la suma de sus requerimientos no debe superar el 100% de CPU ni los 768 MB de memoria. Al evaluar todas las combinaciones, se identifican los siguientes pares que superan el límite de hardware:

- **Por exceso de CPU (Suma mayor a 100%):** El par compuesto por P1 y P2 requiere un 110% de CPU (50% de P1 más 60% de P2).
- **Por exceso de Memoria (Suma mayor a 768 MB):** Los pares compuestos por P1 y P3 (requieren 800 MB), P3 y P4 (requieren 800 MB), y P3 y P5 (requieren 900 MB).

Por lo tanto, los pares de procesos que no pueden compartir una misma CPU bajo ninguna circunstancia son: **{P1, P2}**, **{P1, P3}**, **{P3, P4}** y **{P3, P5}**.

c) **Asignación válida con menor tráfico en la red:**

Para encontrar la distribución óptima se deben descartar todas las opciones de la tabla anterior que junten a los pares prohibidos en el inciso b (es decir, las parejas P1-P2, P1-P3, P3-P4 y P3-P5). Al filtrar las combinaciones de menor tráfico que sí cumplen con los límites de hardware de 100% de CPU y 768 MB de memoria, se determina que la solución óptima tiene un tráfico de red de 20.

La distribución física de los procesos en los procesadores debe quedar de la siguiente manera:

- **CPU-1:** Aloja a los procesos P1 y P4. Este grupo requiere un 90% de CPU (50% más 40%) y 600 MB de memoria (300 MB más 300 MB), manteniéndose dentro de los límites.
- **CPU-2:** Aloja a los procesos P2 y P3. Este grupo requiere un 90% de CPU (60% más 30%) y 700 MB de memoria (200 MB más 500 MB), cumpliendo también con las restricciones.
- **CPU-3:** Aloja únicamente al proceso P5, el cual requiere un 40% de CPU y 400 MB de memoria.

6) a) Al fijar obligatoriamente al proceso P4 en la CPU-3, debemos revisar en nuestra tabla de asignaciones cuáles opciones cumplen con esta condición. La CPU-3 tiene una capacidad máxima de 1 proceso, por lo que P4 debe quedarse completamente solo en ese procesador.

Al buscar las opciones donde la CPU-3 es igual a P4, encontramos las opciones 3 y 4. Ambas presentan un tráfico de red de 15. Por lo tanto, la asignación óptima es la siguiente:

- **CPU-1:** Aloja a los procesos P1 y P2.
- **CPU-2:** Aloja a los procesos P3 y P5.
- **CPU-3:** Aloja al proceso P4 de forma exclusiva.
- **Tráfico de Red Total:** 15.

b) Si la restricción cambia y ahora es el proceso P3 el que debe ejecutarse de forma obligatoria y exclusiva en la CPU-3 (debido a su capacidad máxima de 1 proceso), revisamos la tabla para buscar las opciones donde la CPU-3 es igual a P3.

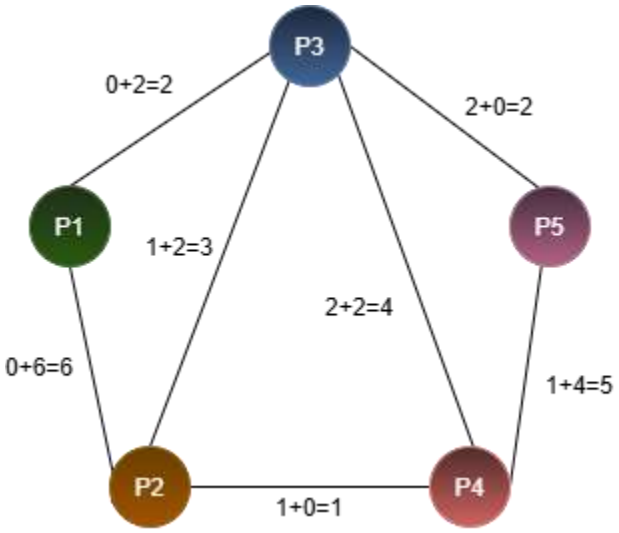
Esto nos conduce directamente a las opciones 5 y 6, las cuales coinciden exactamente con la solución óptima absoluta que ya habíamos encontrado en el ejercicio 4.a. Por lo tanto, la asignación óptima es la siguiente:

- **CPU-1:** Aloja a los procesos P1 y P2.
- **CPU-2:** Aloja a los procesos P4 y P5.
- **CPU-3:** Aloja al proceso P3 de forma exclusiva.
- **Tráfico de Red Total:** 12.

c) La restricción del inciso a aumenta el tráfico de red (pasa de 12 a 15) porque obliga a separar físicamente a los procesos P4 y P5. En la tabla de datos base, la conexión entre P4 y P5 es una de las más pesadas del sistema (vale 5). Al mandar a P4 a la CPU-3 de forma solitaria, esa comunicación se ve forzada a salir por el cable de red externo, incrementando el tráfico global.

Por el contrario, la restricción del inciso b no aumenta el tráfico porque el proceso P3, por naturaleza del diseño óptimo, ya debía estar solo en la CPU-3 para lograr el mínimo impacto en la red. Colocar a P3 en la CPU-3 permite que las parejas con mayor comunicación mutua (P1-P2 que vale 6 y P4-P5 que vale 5) permanezcan unidas localmente dentro de la CPU-1 y la CPU-2, manteniendo el tráfico de red en su mínimo posible de 12.

7) a)



La operación que hay que hacer es la suma del tráfico en ambos sentidos para cada par de procesos (Tráfico de X hacia Y más Tráfico de Y hacia X).

b) Dado que al sumar ambos sentidos de comunicación obtenemos exactamente los mismos valores netos que en los datos base originales, la asignación óptima se mantiene idéntica respetando las capacidades máximas de hardware (CPU-1 admite 2 procesos, CPU-2 admite 2 procesos y CPU-3 admite 1 proceso).

La distribución óptima que minimiza el tráfico de red es:

- **CPU-1:** Aloja a los procesos P1 y P2.
- **CPU-2:** Aloja a los procesos P4 and P5.
- **CPU-3:** Aloja únicamente al proceso P3.

Esta asignación genera un tráfico mínimo en la red de **12 KB/s**.

c) Si evaluamos la distribución propuesta por el compañero (CPU-1 = {P2, P4}, CPU-2 = {P3, P5} y CPU-3 = {P1}) utilizando los valores de tráfico reales (la suma de ambos sentidos), el cálculo se realiza de la siguiente manera:

- **Enlaces locales (comunicación interna dentro de la misma CPU):**
  - En la CPU-1 {P2, P4}: El enlace real P2-P4 tiene un valor de 1 KB/s (1 de ida más 0 de vuelta).
  - En la CPU-2 {P3, P5}: El enlace real P3-P5 tiene un valor de 2 KB/s (2 de ida más 0 de vuelta).
  - En la CPU-3 {P1}: Al haber un solo proceso, no existen enlaces internos.
  - Suma total de tráfico local: 1 KB/s + 2 KB/s = 3 KB/s.
- **Cálculo del Tráfico de Red Real:**
  Sabiendo que el tráfico total absoluto de todo el sistema es de 23 KB/s, le restamos la comunicación interna para conocer cuántos datos viajan obligatoriamente por el cable físico:
  23 KB/s (Total) - 3 KB/s (Local) = 20 KB/s.

En realidad, esa asignación genera un tráfico de **20 KB/s**. El compañero cometió un error conceptual al ignorar por completo la columna de retorno (Y hacia X), lo que lo llevó a elegir una distribución ineficiente para el sistema real.

8) **a) Calcule T en cada estación:**

En el modelo de estaciones de trabajo independientes, cada máquina funciona de manera aislada como un sistema de colas simple (M/M/1). La fórmula para calcular el tiempo de respuesta total (T) es: $T = 1 / (\mu - \lambda)$

Sustituyendo los valores individuales:
$T = 1 / (20 - 15)$

T = 1 / 5
T = 0.2 segundos (equivalente a 200 milisegundos)

El tiempo de respuesta en cada estación individual es de **0.2 segundos**.

b)**Calcule T1 si se juntan las 4 CPU en una pila:**

Al unificar las 4 CPUs en una pila de procesadores con una única cola centralizada, los recursos y las cargas se consolidan. La tasa de llegada total del sistema se multiplica por cuatro (4 multiplicado por λ) y la capacidad de servicio total de la pila también se multiplica por cuatro (4 multiplicado por μ).

La fórmula para el nuevo tiempo de respuesta (T1) es: $T1 = 1 / ((n * \mu) - (n * \lambda))$

Sustituyendo los valores agrupados:
T1 = 1 / ((4 * 20) - (4 * 15))
T1 = 1 / (80 - 60)
T1 = 1 / 20
T1 = 0.05 segundos (equivalente a 50 milisegundos)

Si se juntan las 4 CPUs en una pila, el nuevo tiempo de respuesta T1 se reduce a **0.05 segundos**.

c) **¿Cuántas veces mejora?**

Para determinar cuántas veces mejora el sistema, calculamos la relación entre el tiempo de respuesta inicial (T) y el tiempo de respuesta optimizado en pila (T1):

Mejora = T / T1
Mejora = 0.2 / 0.05
Mejora = 4 veces

El sistema mejora exactamente **4 veces** (el tiempo de respuesta se vuelve 4 veces más rápido). Esto demuestra el principio de que una sola cola compartida para múltiples servidores aprovecha mucho mejor la capacidad de cómputo que tener colas individuales separadas.

9) a) **Calcule T y T1:**

- **Cálculo de T (Modelo de estaciones individuales aisladas):**
  Cada máquina opera como una cola independiente M/M/1.
  $T = 1 / (\mu - \lambda)$

T = 1 / (10 - 9.5)
T = 1 / 0.5
T = 2 segundos (equivalente a 2000 milisegundos)

- **Cálculo de T1 (Modelo de pila de procesadores centralizada):**
  Al unificar los recursos, la capacidad total de procesamiento sube a 100 (10 por 10) y la tasa de llegada total a la gran cola única sube a 95 (10 por 9.5).
  T1 = 1 / ((n * μ) - (n * λ))
  T1 = 1 / ((10 * 10) - (10 * 9.5))
  T1 = 1 / (100 - 95)
  T1 = 1 / 5
  T1 = 0.2 segundos (equivalente a 200 milisegundos)

  El tiempo de respuesta individual T es de **2 segundos**, mientras que el tiempo en la pila de procesadores T1 se reduce a **0.2 segundos**.

  b) **b. Compare con el caso λ = 8:**

  Para realizar la comparación, calculamos primero los valores teóricos del caso clásico dictado en clase donde la carga es menor (λ = 8 trabajos/s):

- **Tiempos con λ = 8:**

1. T = 1 / (10 - 8) = 1 / 2 = 0.5 segundos.
2. T1 = 1 / (100 - 80) = 1 / 20 = 0.05 segundos.

- **Análisis comparativo:**

1. **Factor de mejora constante:** En ambos niveles de carga (tanto con λ = 8 como con λ = 9.5), la pila de procesadores reduce el tiempo de respuesta exactamente **10 veces**, lo cual coincide con la cantidad de servidores unificados (n = 10).
2. **Sensibilidad ante la saturación:** Cuando el sistema trabaja de forma independiente con una carga del 95% (λ = 9.5), el tiempo de respuesta se dispara de forma crítica pasando de 0.5 segundos a 2 segundos (se cuadruplica), volviéndose sumamente ineficiente.
3. **Ventaja de la centralización:** Al utilizar el modelo de pila de procesadores con la carga alta (λ = 9.5), se logra un tiempo de respuesta de tan solo **0.2 segundos**. Esto significa que la pila saturada al 95% responde incluso más rápido que una estación de trabajo individual que opera con una carga mucho menor del 80% (0.5 segundos).

  Aunque en ambos casos la mejora relativa es de 10 veces, la comparación demuestra que la pila de procesadores es drásticamente más robusta ante aumentos extremos de carga,

evitando el colapso por tiempos de espera que sufren las máquinas individuales cuando se acercan a su límite de capacidad.

TRABAJO PRÁCTICO N°8: SISTEMAS DISTRIBUIDOS DE ARCHIVOS

1) a) 3,375 ms

b) la tasa de acierto necesaria es del 86,96%

2) a) 4,04 s

b) 3,232 s

c) tardaría 0,352 s

3) a) Como el archivo se lee con mucha más frecuencia de la que se escribe, nos conviene elegir un Nr bien chico para que las operaciones de lectura sean rápidas y requieran menos servidores, compensando con un Nw más alto. Proponemos los siguientes valores: Nr = 1 y Nw = 12

Para que los valores sean válidos en el algoritmo del voto de Gifford, se tienen que cumplir las reglas de solapamiento:

Nr + Nw > N

- 1 + 12 = 13, y como 13 > 12, se cumple.

Nw > N / 2

- 12 > 12 / 2 (12 > 6), se cumple.

Por lo tanto, los valores propuestos son válidos y están optimizados para un sistema con alta frecuencia de lectura.

b) No, no es válido. Para que los valores sean válidos en el algoritmo del voto de Gifford, se debe cumplir la regla de solapamiento para evitar lecturas de datos obsoletos: Nr + Nw > N.

Si tomamos los valores propuestos por el compañero (Nr = 6 y Nw = 6) con un total de N = 12 servidores:

• Nr + Nw = 6 + 6 = 12

• Como 12 no es mayor estricto que 12 (12 > 12 es falso), la regla no se cumple.

Ejemplo: Imaginemos que se realiza una escritura exitosa contactando a 6 servidores. Si luego intentamos hacer una lectura y el quórum de lectura de 6 servidores (Nr = 6) elige una combinación completamente distinta de 6 servidores que no participaron en la última escritura, nos encontraremos con que ningún servidor del grupo de lectura posee la versión más actualizada del archivo. Por lo tanto, se podría leer un dato viejo, lo cual invalida la configuración.

c) **Con el par propuesto en a), ¿cuántos servidores pueden fallar sin impedir las lecturas?**

Como el quórum de lectura es Nr = 1, el sistema necesita que responda al menos 1 servidor para poder leer. Por lo tanto, pueden fallar hasta 11 servidores sin impedir las lecturas (12 - 1 = 11).

¿Y sin impedir las escrituras?

Como el quórum de escritura es Nw = 12, se necesita la participación y respuesta de todos los servidores del sistema para completar una escritura. Por lo tanto, no puede fallar ningún servidor (0 servidores) sin impedir las escrituras (12 - 12 = 0).

4) Espacio ocupado por el contenido real:

Cantidad de archivos: 2.000.000

Tamaño promedio: 3 KB

Cálculo: 2.000.000 * 3 KB = 6.000.000 KB (o 6 GB)

_Espacio ocupado en disco:

Como cada archivo ocupa al menos un bloque de 4 KB para su contenido: 2.000.000 * 4 KB = 8.000.000 KB (8 GB)

Espacio destinado a metadatos (256 bytes por archivo): 2.000.000 * 256 bytes = 512.000 KB (0,512 GB)

Cálculo total en disco: 8.000.000 KB + 512.000 KB = 8.512.000 KB (8,512 GB)

_Espacio desperdiciado:

Fragmentación interna (1 KB desperdiciado por archivo entre el tamaño real y el bloque de 4 KB): 2.000.000 * 1 KB = 2.000.000 KB

Incluyendo los metadatos como sobrecarga del sistema: 2.000.000 KB + 512.000 KB = 2.512.000 KB (2,512 GB)

_Eficiencia de almacenamiento:

Fórmula: (Espacio de contenido real / Espacio total en disco) * 100

Cálculo: (6.000.000 / 8.512.000) * 100 = 70,49%

5) a) **Si el servicio está disponible cuando funciona al menos un servidor, calcule su disponibilidad.**

_Probabilidad de que un servidor falle: q = 1 - 0,90 = 0,10

_Probabilidad de que fallen los 3 servidores al mismo tiempo: (0,10)^3 = 0,001
_Disponibilidad del sistema (que funcione al menos uno): 1 - 0,001 = 0,999 (es decir, 99,9%)

b) **¿Cuántos servidores harían falta como mínimo para llegar al 99,999 %?**

_La fórmula para la disponibilidad con al menos un servidor operativo es: 1 - (0,10)^N >= 0,99999 Despejando, necesitamos que la probabilidad de fallo conjunto sea menor o igual a 0,00001 (10^-5).

_Como (0,10)^5 = 0,00001, se necesitan como mínimo 5 servidores.

c) **Si, en cambio, el servicio necesitara que los 3 servidores funcionen a la vez, ¿cuál sería su disponibilidad?**

_Probabilidad de que los 3 funcionen simultáneamente: 0,90 * 0,90 * 0,90 = (0,90)^3 = 0,729

_Disponibilidad resultante: 0,729 (es decir, 72,9%)

6) a) **¿Qué versión obtiene B en cada lectura?**

_En la primera lectura (t = 25 s), el cliente B obtiene la versión antigua o anterior del archivo, ya que el cliente A modificó su caché en t = 12 s pero todavía no sincronizó con el servidor (la primera sincronización programada ocurre recién en t = 30 s).

_En la segunda lectura (t = 40 s), el cliente B obtiene la versión nueva o actualizada, porque el cliente A ya envió sus modificaciones al servidor en el intervalo de los 30 segundos (en t = 30 s).

b) **¿Cuánto dura la inconsistencia para esta modificación? ¿Cuál es la ventana máxima posible con este intervalo?**

_Para este caso específico, la modificación se hizo en t = 12 s y el servidor se actualizó en t = 30 s, por lo que la inconsistencia duró 18 segundos (desde t = 12 s hasta t = 30 s).

_La ventana máxima posible de inconsistencia con este intervalo de 30 segundos es de 30 segundos justos (por ejemplo, si una modificación ocurre justo después de una sincronización, el cambio recién viaja en la siguiente sincronización a los 30 segundos).

c) **A realiza 10 escrituras por segundo sobre el archivo. ¿Cuántos mensajes de escritura recibe el servidor por minuto con escritura a través del caché y cuántos con escritura retrasada?**

_Con escritura a través del caché (write-through), cada escritura que hace el cliente se envía de inmediato al servidor. Como A hace 10 escrituras por segundo y un minuto tiene 60 segundos, el servidor recibe 600 mensajes por minuto (10 por 60).

_Con escritura retrasada (write-back), los cambios se acumulan y se envían cada 30 segundos independientemente de cuántas veces modifique el archivo el cliente. Por lo tanto, en un minuto (que abarca dos intervalos de envío: a los 30 s y a los 60 s), el servidor recibe solo 2 mensajes de actualización en total.